FieldTracking

AWS Hosting Guide - Step by Step

This guide walks you through hosting the FieldTracking platform (Django API, Next.js dashboard, PostgreSQL/TimescaleDB and Redis) on a single Amazon EC2 virtual machine using Docker Compose, with an Nginx reverse proxy serving everything on one address. No prior AWS or DevOps experience is assumed.

Estimated time: about 15-20 minutes. Estimated cost: a small instance runs roughly a few US dollars per month (free-tier eligible for the first year on a t2/t3.micro, though 4 GB RAM is recommended).

This is the HTTP-on-public-IP setup you selected. Section 11 shows how to upgrade to a domain + HTTPS, which is strongly recommended before real use.

1. What you will build

Everything runs on one EC2 instance. A single program called Nginx listens on the public web port (80) and quietly forwards each request to the right place:

- Requests for the dashboard pages go to the Next.js web app.

```
                +------------------ EC2 instance ------------------+
Browser  --80-->|  Nginx --> Next.js (web)                        |
Mobile   --80-->|        \-> Django/Daphne (api) --> PostgreSQL   |
                |                                  \-> Redis       |
                +-------------------------------------------------+
```

Because the browser only ever talks to one address, there are no cross-origin problems, and the server's IP is never hard-coded into the website - so you can move servers or add a domain later without rebuilding anything.

Prerequisites

- An AWS account (https://aws.amazon.com).

2. Launch the server (EC2 instance)

1. Sign in to the AWS Console and open the EC2 service. Click 'Launch instance'.
2. Name it 'fieldtracking'.
3. Application and OS image (AMI): choose 'Ubuntu Server 24.04 LTS' (64-bit x86).
4. Instance type: choose 't3.medium' (4 GB RAM, recommended). 't3.small' works for a light demo; the free-tier 't2.micro' is usable but tight.
5. Key pair: click 'Create new key pair', give it a name, and download the .pem file. You will need it to log in - keep it safe.
6. Network settings: click 'Edit', then under 'Firewall (security groups)' add the inbound rules in the table below.
7. Configure storage: set 30 GB (gp3).
8. Click 'Launch instance', then 'View all instances'.

Inbound firewall rules (security group)

Type	Port	Source	Why
SSH	22	My IP	Admin login (you only)
HTTP	80	Anywhere 0.0.0.0/0, ::/0	The app, for everyone

Do NOT open ports 5432, 6379, 8000 or 3000. The database, Redis and app containers stay private and must not

be reachable from the internet.

3. Give it a permanent IP (Elastic IP)

By default a server's public IP changes if you ever stop and start it. An Elastic IP pins a fixed address so your links and the mobile app keep working.

1. In the EC2 console left menu, open 'Elastic IPs' (under Network & Security).
2. Click 'Allocate Elastic IP address' -> 'Allocate'.
3. Select the new address -> 'Actions' -> 'Associate Elastic IP address'.
4. Choose your 'fieldtracking' instance and click 'Associate'.

Write down this address - this guide calls it EIP (for example 13.234.56.78). Wherever you see EIP below, type your real Elastic IP.

4. Connect to the server (SSH)

Open a terminal (PowerShell on Windows, Terminal on macOS/Linux) in the folder where you saved the .pem key file, then run:

```
ssh -i your-key.pem ubuntu@EIP
```

On Windows, if it complains the key is too open, run this once first:

```
icacls.exe your-key.pem /inheritance:r /grant:r "%USERNAME%:R"
```

Type 'yes' when asked to trust the host. You are now logged into the server.

5. Install Docker

Docker runs the application's containers. Paste this whole block into the SSH session:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-compose-plugin
sudo usermod -aG docker $USER && newgrp docker
```

Verify it works (should print a version number):

```
docker --version && docker compose version
```

6. Put the project on the server

Pick ONE of these options.

Option A - from a Git repository (if you pushed the code to GitHub)

```
git clone <your-repository-url> fieldtracking
cd fieldtracking
```

Option B - copy from your computer

Run this on YOUR computer (not the server), from the project's parent folder. It securely copies the files up:

```
scp -i your-key.pem -r "fieldtracking - Copy" ubuntu@EIP:~/fieldtracking
```

Then back in the SSH session on the server:

```
cd ~/fieldtracking
```

7. Configure secrets (.env)

Create your settings file from the template and open it in a text editor:

```
cp .env.production.example .env
nano .env
```

Fill in these values, then save (in nano: Ctrl+O, Enter, Ctrl+X):

- POSTGRES_PASSWORD - a strong database password.
- DATA
  (postg

Generate the two random secrets with this command and paste the output:

```
python3 -c "import secrets; print(secrets.token_urlsafe(50))"
```

8. Build and start the application

This one command builds the images and starts every service in the background. The first run also sets up the database tables and (if enabled) demo data:

```
docker compose -f docker-compose.prod.yml up -d --build
```

Check that all five services are running:

```
docker compose -f docker-compose.prod.yml ps
```

Watch the backend start up (press Ctrl+C to stop watching - it keeps running):

```
docker compose -f docker-compose.prod.yml logs -f api
```

9. Open the app

In a browser, go to:

```
http://EIP
```

Interactive API documentation is at http://EIP/api/docs/

If demo data was seeded, log in with password Password123! using any of:

Email	Role	Lands on
owner@fieldtracking.dev	Platform owner	Owner portal
admin@acme.test	Tenant admin	Dashboard
dispatch@acme.test	Dispatcher	Dashboard
ravi@acme.test	Field staff	Mobile app

For a real deployment, set SEED_DEMO=0 (or change these passwords) so the demo accounts do not exist on your live system.

10. Point the mobile app at the server

Rebuild the Android APK so it talks to your server. Note the address is the Elastic IP on port 80 with the /api/v1 path (Nginx routes it):

```
docker run --rm -v "$PWD/apps/mobile:/work" \
  -v flutter-android-sdk:/opt/android-sdk-linux \
  -v flutter-gradle-cache:/root/.gradle \
  -w /work ghcr.io/cirruslabs/flutter:stable \
  bash /work/build_apk.sh http://EIP/api/v1
```

Install the resulting APK from apps/mobile/build/app/outputs/flutter-apk/ onto an Android phone and log in as a field-staff user. The phone needs internet access to reach the server's public IP.

11. Running it day to day

View logs

```
docker compose -f docker-compose.prod.yml logs -f
```

Restart or stop

```
docker compose -f docker-compose.prod.yml restart
docker compose -f docker-compose.prod.yml down
```

Deploy a new version

```
git pull            # or copy the new code up again
docker compose -f docker-compose.prod.yml up -d --build
```

Back up the database

```
docker compose -f docker-compose.prod.yml exec -T db \
  pg_dump -U fieldtracking fieldtracking | gzip > backup-$(date +%F).sql.gz
```

Uploaded task photos and the database live in Docker volumes (mediadata and dbdata) and survive restarts and updates. They are only deleted if you run 'down -v'.

12. Upgrade to a domain + HTTPS (recommended)

Running over plain HTTP is fine for a demo, but real use should be encrypted.

1. Buy a domain and add an 'A record' pointing to your Elastic IP.
2. Add a TLS certificate. The simplest route is a free Let's Encrypt certificate via Certbot, terminated by Nginx on port 443.
3. In .env set SECURE_SSL=1 and CSRF_TRUSTED_ORIGINS=https://your-domain.
4. Re-run: docker compose -f docker-compose.prod.yml up -d --build.
5. Rebuild the APK with https://your-domain/api/v1.

Because the website uses relative links, switching to a domain needs no website rebuild - only DNS, the certificate, and the env flag.

13. Scaling beyond one server (later)

When a single machine is no longer enough, move the stateful parts to managed AWS services and keep the same container images:

- PostgreSQL -> Amazon RDS (or Timescale Cloud) for managed backups/failover.
- Redis

You would simply point DATABASE_URL and REDIS_URL at the managed endpoints; the same api and web images run unchanged.

14. Security checklist

- [ ] Strong, unique POSTGRES_PASSWORD, SECRET_KEY and JWT_SECRET.
- [ ] DE

15. Troubleshooting

The page does not load

- Confirm the security group allows inbound port 80 from anywhere.
- Run '

Admin login says CSRF verification failed

- Set CSRF_TRUSTED_ORIGINS=http://EIP in .env, then rebuild.

Database errors on first start

- The DATABASE_URL password must exactly match POSTGRES_PASSWORD.

Mobile app cannot connect

- Rebuild the APK with the correct http://EIP/api/v1 address.

- If you

- Make